

Agentic Engineering mit Codex & Claude Code

Executive Summary

Die beste Arbeitsweise mit modernen Coding-Agents ist nicht primär **Prompt Engineering**, sondern **Harness Engineering**: Das Repository, die Entwicklungsumgebung, die Tool-Anbindung und die Feedback-Loops werden so gestaltet, dass ein Agent relevante Informationen selbst finden, Änderungen isoliert ausführen, seine Arbeit verifizieren und Fehler systematisch reparieren kann. OpenAI beschreibt diesen Wandel explizit als Verschiebung von „Menschen schreiben Code“ zu „Menschen spezifizieren Intent, entwerfen Umgebungen und bauen Feedback-Loops“. ¹ Anthropic formuliert im aktuellen AI-native SDLC sehr ähnlich: institutionelles Wissen wird zu versionierten Dateien, Guardrails werden zu Code, und nicht-triviale Änderungen beginnen mit Spec und akzeptiertem Plan. ²

Für die praktische Arbeit ergibt sich ein robustes Grundmuster:

1. **Intent/Spec festschreiben.** Was soll sich ändern, was ausdrücklich nicht, welche Constraints und Akzeptanzkriterien gelten?
2. **Read-only Exploration + Plan.** Der Agent liest Repo und interne Doku, benennt betroffene Dateien, Risiken, Alternativen und die konkrete Beweisführung. Hängt die Änderung von externen Frameworks, SDKs, APIs oder Cloud-Services ab, prüft er vor dem Plan zusätzlich die aktuelle offizielle, versionspassende Dokumentation.
3. **Kontrolliert implementieren.** Nicht-triviale Änderungen werden mit kleinem Scope, sauberem Git-Diff und klarer Reversibilität umgesetzt.
4. **Feedback-Loops schließen.** Tests, Typecheck, Lint, Build, E2E, Screenshots, Logs oder Metriken müssen vom Agenten selbst ausführbar sein.
5. **Unabhängig reviewen.** Review gegen Spec, Plan, Architekturregeln und Sicherheitsinvarianten — nicht nur „Tests grün“.
6. **Aus Fehlern Harness-Assets machen.** Wiederkehrende Korrekturen werden zu Test, Linter, Rule, Skill, Hook, Doku oder Eval; nicht zu immer längeren Ad-hoc-Prompts.

Die Toolwahl zwischen Codex und Claude Code ist inzwischen weniger wichtig als die Qualität dieses Systems. Claude Code hat aktuell besonders ausgeprägte eingebaute Primitive für Regeln, Hooks und Skills. Codex ist besonders interessant, wenn der Agent-Loop in bestehende Produkte oder CI-Jobs eingebettet werden soll: OpenAI veröffentlicht CLI, SDK und App-Server als offene Harness-Komponenten. Beide unterstützen aber dieselben Kernprinzipien.

Empfohlene Strategie: Einen primären Agenten zum Implementieren verwenden, den anderen selektiv als unabhängigen Reviewer oder für alternative Lösungsentwürfe einsetzen. Welcher Agent welche Aufgabentypen übernimmt, sollte über ein kleines repo-spezifisches Eval-Set entschieden werden — nicht über pauschale Modellpräferenzen.

¹ Ryan Lopopolo, OpenAI, “Harness engineering: leveraging Codex in an agent-first world,” 11 Feb. 2026. <https://openai.com/index/harness-engineering/>

² Anthropic, “The AI-Native SDLC playbook,” Aug. 2026. <https://claude.com/blog/the-ai-native-sdlc-playbook>

1. Das eigentliche Problem: Agenten brauchen ein gutes System, nicht nur gute Prompts

OpenAIs „Harness Engineering“-Erfahrung ist derzeit eine der stärksten Fallstudien für agent-first Softwareentwicklung. In einem internen Produkt wurden über fünf Monate praktisch alle Artefakte — Anwendungscode, Tests, CI, Doku, Observability und Tooling — von Codex erzeugt. Die zentrale Lektion war nicht, dass ein langer Systemprompt nötig ist, sondern dass Agenten **legible** Systeme brauchen: navigierbare Repo-Struktur, explizite Constraints, direkt zugängliche Logs/Metriken, reproduzierbare Dev-Umgebungen und mechanisch überprüfbare Architekturregeln.³

Das führt zu einer nützlichen Hierarchie:

Problemtyp	Richtige Ebene	Beispiel
Immer gültiges Repo-Wissen	AGENTS.md / CLAUDE.md	Build-Kommandos, Architekturkarte, kritische Invarianten
Pfad-/Komponenten-spezifische Regel	Nested instructions / path rules	API-Handler müssen Eingaben validieren
Wiederholbarer Prozess	Skill / Command	Release-Check, Migration, PR-Summary
Deterministischer Zwang	Hook / Script / Linter / CI	Formatter, verbotener Befehl, Architekturlayer
Task-spezifisches Ziel	spec.md / Issue	Feature, Bugfix, Nicht-Ziele, Acceptance Criteria
Umsetzung eines komplexen Tasks	plan.md	Dateien, Reihenfolge, Risiken, Tests
Externe Systeme/Aktionen	MCP / native Tools	Jira/Linear, DB, Browser, Observability

Der entscheidende Designgrundsatz ist **progressive disclosure**: Die immer geladene Anweisung bleibt klein; Details werden erst geladen, wenn sie relevant sind. OpenAI berichtet, dass eine monolithische AGENTS.md schnell Kontext verdrängt, veraltet und schwer verifizierbar wird, und nutzt sie stattdessen als Inhaltsverzeichnis in eine strukturierte docs/-Wissensbasis.⁴ Anthropic empfiehlt dasselbe für CLAUDE.md: root-level Überblick, lokale Dateien für lokale Konventionen, Skills für on-demand Expertise.⁵

Praktische Regel

Wenn eine Anweisung zuverlässig als Code ausgedrückt werden kann, sollte sie kein Prompt bleiben.

„Führe nach jeder Änderung den Linter aus“ ist schwächer als ein Hook oder CI-Check. „Benutze niemals Dependency X in Layer Y“ ist schwächer als ein struktureller Linter. Prompts sollten dort eingesetzt werden, wo tatsächlich Urteilskraft und Kontext nötig sind.

2. Empfohlene Repository-Struktur für Codex + Claude Code

Für ein Repo, das beide Tools gut unterstützt, sollte die gemeinsame Wissensbasis **tool-neutral** sein. Tool-spezifische Dateien bleiben dünne Adapter.

³ Ryan Lopopolo, OpenAI, “Harness engineering: leveraging Codex in an agent-first world,” 11 Feb. 2026. <https://openai.com/index/harness-engineering/>

⁴ Ryan Lopopolo, OpenAI, “Harness engineering: leveraging Codex in an agent-first world,” 11 Feb. 2026. <https://openai.com/index/harness-engineering/>

⁵ Anthropic, “How Claude Code works in large codebases: Best practices and where to start,” 2026. <https://claude.com/blog/how-claude-code-works-in-large-codebases-best-practices-and-where-to-start>

```

repo/
├── AGENTS.md           # Codex-Adapter: Repo-Karte, Commands, kritische Regeln
├── CLAUDE.md           # Claude-Adapter: Repo-Karte, Commands, kritische Regeln
├── ARCHITECTURE.md     # aktueller Systemzustand: Komponenten, Layer, Datenflüsse
├── docs/
│   ├── product-specs/  # fachliches „Was“: Features, Verhalten, Akzeptanzkriterien
│   ├── design-decisions/ # „Warum so?“: Architektur-/Designentscheidungen (ADRs)
│   └── exec-plans/     # „Wie ändern wir es?“: konkrete Task-/Migrationspläne
│       ├── active/
│       └── completed/
├── agent/
│   ├── workflows/     # tool-neutrale, wiederkehrende Agent-Abläufe
│   └── scripts/        # deterministische Checks und Helper
├── .agents/skills/     # dünne Codex-Wrapper auf gemeinsame Workflows
├── .claude/skills/     # dünne Claude-Wrapper auf gemeinsame Workflows
├── .claude/rules/      # Claude: pfad-/komponentenspezifische Regeln
└── .github/workflows/  # CI und verpflichtende deterministische Checks
    
```

Was gehört konkret wohin?

Die Struktur trennt bewusst sechs Arten von Wissen: Orientierung, aktuellen Systemzustand, fachliche Anforderungen, historische Entscheidungen, konkrete Ausführungspläne und ausführbare Agent-Mechanik. Diese Trennung verhindert, dass dieselbe Information an mehreren Stellen unterschiedlich gepflegt wird.

Root-Dateien: Orientierung und aktueller Systemzustand

AGENTS.md — ist der dünne Einstiegspunkt für Codex. Er enthält nur das, was Codex in praktisch jeder Session sofort wissen muss: Repo-Karte, Standardkommandos, wenige kritische Invarianten, Definition von „done“ und Links auf die kanonische Doku. Keine langen Architekturklärungen oder Task-spezifischen Pläne hier duplizieren.

CLAUDE.md — erfüllt für Claude Code dieselbe Rolle. Tool-spezifische Hinweise dürfen hier stehen, die fachliche und architektonische Wahrheit aber nicht separat von AGENTS.md gepflegt werden. Beide Dateien sollten auf dieselben docs/ und agent/-Artefakte zeigen.

ARCHITECTURE.md — beschreibt den aktuellen Soll-/Ist-Zustand des Systems: Hauptkomponenten, Layer, Abhängigkeitsrichtung, zentrale Datenflüsse, externe Systeme und wichtige Einstiegspunkte im Code. Sie beantwortet „Wie ist das System heute aufgebaut?“, nicht „Warum wurde Entscheidung X vor acht Monaten getroffen?“ — dafür gibt es design-decisions/.

docs/: kanonisches Produkt-, Architektur- und Task-Wissen

docs/product-specs/ — enthält langlebigere fachliche Spezifikationen. Eine Spec beschreibt Verhalten aus Nutzer-/Systemsicht: Goal, Non-goals, Constraints, Schnittstellen und Acceptance Criteria. Sie sollte möglichst unabhängig davon sein, wie der Code intern umgesetzt wird. Für einen einzelnen kleinen Task kann auch eine task-spezifische spec.md genügen; product-specs/ ist für Spezifikationen gedacht, die als dauerhafte Referenz relevant bleiben.

docs/design-decisions/ — enthält kurze Architecture/Design Decision Records. Hier steht eine Entscheidung mit Kontext, betrachteten Alternativen, gewählter Lösung und Konsequenzen. Das ist die Historie des „Warum“. ARCHITECTURE.md zeigt dagegen nur den aktuellen Zustand und muss nicht jede verworfene Alternative erklären.

docs/exec-plans/ — enthält konkrete, task-spezifische Arbeitsdateien für nicht-triviale Änderungen. Sie verbinden Plan und Continuity: betroffene Dateien, Reihenfolge, Risiken, Trade-offs, Verification und Rollback stehen zusammen mit dem aktuellen Fortschritt, getroffenen Entscheidungen, verworfenen Ansätzen, Learnings und dem nächsten Schritt. **active/** enthält laufende Vorhaben; nach Abschluss wandert die Datei nach **completed/**. So kann eine neue Session einen Task fortsetzen, ohne den vorherigen Chat zu benötigen. Die Datei beschreibt das konkrete „Wie“ und den temporären Task-State — nicht die dauerhafte Produkthanforderung.

Für sequenzielles Arbeiten ist eine Datei pro Task meist einfacher als getrennte plan.md-, progress.md- und continuity.md-Dateien:

```
feature-x.md
```

```
# Goal
```

```

...

# Plan
...

# Trade-offs / Decisions
...

# Progress
...

# Failed approaches
...

# Learnings
...

# Next step
...
    
```

Während des Tasks sammelt # Learnings beobachtete Erkenntnisse und korrigierte Fehlannahmen, ohne sie sofort zu globalen Regeln zu machen. Beim Abschluss wird bewusst getrennt: reine Task-Historie bleibt im abgeschlossenen Exec-Plan; bestätigte und generalisierbare Learnings werden in die passende dauerhafte Form „promoted“ — z. B. ARCHITECTURE.md, eine Design Decision, AGENTS.md/CLAUDE.md, einen Test/Linter/CI-Check oder einen wiederkehrenden Workflow. So entsteht ein Feedback-Loop, ohne dass eine globale learn.md zu einer ungeprüften zweiten Wahrheit wird.

agent/: das gemeinsame, tool-neutrale Harness

agent/workflows/ — enthält kanonische Prozessbeschreibungen, die Codex und Claude gleichermaßen nutzen sollen. Beispiele: plan-from-spec, debug-fix oder independent-review. Ein Workflow beschreibt Reihenfolge, Quellen, Entscheidungsregeln und erwarteten Output. Er ist absichtlich tool-neutral; .agents/skills und .claude/skills sind nur Adapter darauf.

agent/scripts/ — enthält ausführbare, deterministische Helper: z. B. einen einheitlichen Fast-Test-Runner, Architekturchecks, Fixtures, Diff-Checks oder ein Script zum Sammeln relevanter Logs. Alles, was zuverlässig in Code ausgedrückt werden kann, ist hier bzw. in normalen Projekt-Tools besser aufgehoben als in einem Prompt.

Tool-spezifische Adapter und Enforcement

.agents/skills/ — enthält nur Codex-spezifische Skill-Metadaten bzw. dünne Wrapper. Die eigentliche Prozesslogik sollte möglichst in agent/workflows/ liegen, damit sie nicht parallel für zwei Tools gepflegt werden muss.

.claude/skills/ — ist das Claude-Code-Pendant. Auch hier besser auf gemeinsame Workflows, Scripts und Doku verweisen, statt eine zweite Version des Prozesses zu schreiben.

.claude/rules/ — ist für Regeln gedacht, die nur in bestimmten Pfaden oder Komponenten gelten, etwa strengere Vorgaben für migrations/, api/ oder security-kritische Module. Globale Regeln gehören nicht unnötig hierhin; mechanische Regeln sollten nach Möglichkeit durch Code erzwungen werden.

.github/workflows/ — ist die letzte, deterministische Schranke: Tests, Lint, Typecheck, Build, Security-Scans und Architekturtests. Was für einen Merge zwingend gelten muss, sollte hier erzwungen werden und nicht davon abhängen, ob der Agent eine Prompt-Regel beachtet.

Die wichtigste Abgrenzung in einem Satz

Spec — fragt: „Was soll das Produkt/System tun?“

Design Decision — fragt: „Warum haben wir diese technische Richtung gewählt?“

ARCHITECTURE.md — fragt: „Wie ist das System aktuell aufgebaut?“

Exec Plan — fragt: „Wie führen wir diese konkrete Änderung sicher aus?“

Agent Workflow — fragt: „Wie soll ein Coding-Agent diesen wiederkehrenden Engineering-Prozess durchführen?“

Für kleine Repositories muss nicht jeder Ordner ab Tag 1 existieren. Die Struktur oben ist bewusst als schlanker Startpunkt gedacht. Runbooks, eine eigene `security.md`, `reliability.md` und `agent/evals/` würde ich erst ergänzen, wenn ein wiederkehrender Bedarf sichtbar wird: Runbooks bei wiederkehrenden Betriebs-/Recovery-Abläufen, `security.md` bei umfangreicheren repo-weiten Security-Invarianten, `reliability.md` bei expliziten SLO-/Resilience-Anforderungen und Evals erst dann, wenn Agent-Workflows systematisch verglichen oder gegen Regressionen abgesichert werden sollen.

AGENTS.md und CLAUDE.md: kurz halten

Beide Dateien sollten im Wesentlichen enthalten:

- Repo-Karte: wichtigste Verzeichnisse und Architektur-Einstiegspunkte.
- Standardkommandos: Setup, lokale Tests, Lint, Typecheck, Build.
- 5–15 wirklich kritische Invarianten.
- Routing: Welche Skills/Workflows sind bei welcher Änderung verpflichtend?
- Definition von „done“: relevante Checks bestanden und Ergebnis kurz belegt.
- Verweise auf tieferliegende Doku statt Doku zu duplizieren.

Anthropic empfiehlt für `CLAUDE.md` explizit, unter etwa 200 Zeilen zu bleiben und sie wie Code zu reviewen.⁶ OpenAI beschreibt für Codex ebenfalls kleine `AGENTS.md`-Dateien und nutzt in den Agents-SDK-Repositories `AGENTS.md` als Policy- und Skill-Trigger-Schicht, während die detaillierten Abläufe in Skills liegen.⁷

Wichtig: keine zwei konkurrierenden Wahrheiten

Wenn Codex und Claude Code im selben Repo genutzt werden, sollten Architektur- und Produktregeln nicht separat in `AGENTS.md` und `CLAUDE.md` gepflegt werden. Beide zeigen auf dieselben kanonischen Dateien in `docs/`. Tool-spezifisch bleiben nur Bedienungsdetails, z. B. „verwende Skill X“ oder „nutze Hook Y“.

Persönliche Defaults: Arbeitsstil statt Projektwahrheit

Persönliche Präferenzen dürfen den Arbeitsstil des Agenten steuern, sollten aber nicht mit teamweiten Projektregeln vermischt werden. Solche Defaults gehören in die persönliche Tool-Konfiguration bzw. lokale, nicht geteilte Instructions; Architektur, Produktverhalten und verpflichtende Checks bleiben im Repo kanonisch.

Sinnvoller Start-Default für die Planung:

```
When planning, always surface material trade-off decisions explicitly.
For each material trade-off:
- state the realistic options,
- summarize the important pros/cons,
- give a recommendation,
- explain why it fits this repository and task.
Do not invent artificial alternatives; if there is no material trade-off, say so briefly.
```

Damit bleibt die Entscheidung beim Engineer sichtbar, während der Agent weiterhin die technische Exploration übernimmt. Besonders relevant sind Trade-offs bei Architektur, Dependencies, API-Design, Datenmodell, Komplexität, Performance, Security, Migration und Wartbarkeit.

⁶ Michael Segner, Anthropic, “Steering Claude Code: when to use CLAUDE.md, skills, hooks, and subagents,” 18 Jun. 2026. <https://claude.com/blog/steering-claude-code-skills-hooks-rules-subagents-and-more>

⁷ Kazuhiro Sera, OpenAI Developers, “Using skills to accelerate OSS maintenance,” 9 Mar. 2026. <https://developers.openai.com/blog/skills-agents-sdk>

3. Der optimale Task-Lifecycle

Task-Triage — Risk Tiering

Nicht jeder Task braucht den vollständigen Spec→Plan→Review-Prozess. Der Prozess sollte proportional zum Fehlerrisiko sein: kleine, lokale und leicht reversible Änderungen bleiben leichtgewichtig; bei weitreichenden oder schwer reversiblen Änderungen steigt die notwendige Planungs- und Review-Tiefe.

Risikostufe	Typische Änderungen	Mindestprozess
LOW	Kleine, lokale, reversible Änderungen; z. B. Rename, Log-Zeile, kleine UI-/Textkorrektur.	Explore → Implement → relevante Checks ausführen. Keine eigene Spec-/Plan-Datei nötig.
NORMAL	Mehrere Dateien, neue Businesslogik, normale Features oder nicht-triviale Bugfixes.	Spec → Repo-Exploration + Freshness Gate → Plan → Implementierung → Verification.
HIGH	Architektur, Security, Migrationen, Datenmodell, öffentliche APIs, produktionskritisches Verhalten oder schwer reversible Änderungen.	Spec → Exploration + Doku → Plan → menschlicher Plan-Check → Implementierung → Verification → unabhängige frische Review-Session.

Faustregel: Wenn die Einstufung unklar ist oder ein Fehler hohe Folgekosten hätte, eine Stufe höher wählen. Risk Tiering reduziert Prozesskosten; es darf nicht dazu dienen, notwendige Sicherheits- oder Kompatibilitätsprüfungen zu überspringen.

Phase A — Intent / Spec

Ein Agent kann technisch korrekten Code produzieren und trotzdem die falsche Sache bauen. Deshalb sollte jede nicht-triviale Änderung mit einem kleinen Spec-Artefakt starten.

Minimum für spec.md:

```
# Goal
Welches Nutzer-/Systemproblem wird gelöst?

# Non-goals
Was wird ausdrücklich nicht geändert?

# Constraints
Kompatibilität, Security, Performance, APIs, Datenmodell, Plattform.

# Acceptance criteria
Beobachtbare Kriterien; möglichst maschinell prüfbar.

# Relevant references
Issue, Design Doc, bestehende Implementierung, Screenshots, API-Doku.
```

Anthropics aktueller SDLC-Playbook trennt intent.md, spec.md und plan.md ausdrücklich: Die Spec definiert das „Was“ und die Randbedingungen; der Plan entsteht danach aus Codebase-Exploration und definiert das „Wie“.⁸

Phase B — Exploration im Read-only-/Plan-Modus

Der Agent soll vor dem Editieren die relevanten Dateien öffnen, bestehende Muster finden und eine konkrete Implementierungsstrategie schreiben. Der Plan soll betroffene Dateien, Reihenfolge, Risiken, Verification und alle

⁸ Anthropic, “The AI-Native SDLC playbook,” Aug. 2026. <https://claude.com/blog/the-ai-native-sdlc-playbook>

materiellen Trade-off-Entscheidungen sichtbar machen. Für jeden echten Trade-off werden realistische Optionen, relevante Vor-/Nachteile und eine begründete Empfehlung genannt; künstliche Alternativen sollen nicht erfunden werden. OpenAI empfiehlt für größere Codex-Änderungen zunächst Exploration und Plan, bevor Implementierung beginnt.

Freshness Gate — externe Doku vor dem Plan

Bei versionsabhängiger externer Technik sollte internes Modellwissen nie die alleinige Source of Truth sein. Claude Code verfügt über integrierten Webzugriff, und OpenAI's aktuelle Modellguidance empfiehlt Web-Recherche statt Annahmen, wenn Fakten unsicher oder unvollständig sein können.

Zuerst die tatsächlich verwendete Version aus Lockfile, Manifest, IaC oder Runtime-Konfiguration bestimmen. Dann bevorzugt die offizielle Dokumentation genau dieser Version lesen; bei Upgrades zusätzlich Changelog/Migrationsguide.

Neuere APIs oder „bessere“ Patterns dürfen als Alternative geprüft werden, aber nicht ungefragt ein Dependency-Upgrade auslösen.

Relevante Versionen, Links und daraus abgeleitete Annahmen kurz im Plan festhalten, damit Review und spätere Sessions sie nachvollziehen können.

Kein Zwang zum Browsen bei reiner lokaler Businesslogik, kleinen Refactorings oder stabilen Sprachgrundlagen. Die Heuristik lautet: extern + versionssensitiv = Doku prüfen; intern + durch Code/Tests eindeutig = Repo ist die primäre Wahrheit.

Primärquellen: Anthropic Claude Code Docs — <https://code.claude.com/docs/en/features-overview>; OpenAI Model Guidance — <https://developers.openai.com/api/docs/guides/latest-model>

Der Plan sollte enthalten:

- Dateien, die geändert/neu angelegt werden.
- Reihenfolge und Abhängigkeiten.
- Daten-/API-Verträge, die betroffen sind.
- Risikoanalyse: Was könnte brechen?
- Verworfenen Alternativen Design und warum.
- Test-/Verification-Plan.
- Rollback-/Reversibilitätsgedanken bei riskanten Änderungen.

Anthropic empfiehlt, einen Plan so weit zu iterieren, dass ein anderer Engineer ihn ohne die ursprüngliche Unterhaltung umsetzen könnte, und ihn dann als `p1an.md` zu versionieren.⁹

Phase C — Implementierung

Nach akzeptiertem Plan implementiert der Agent den kleinsten sinnvollen Diff. Wichtig ist eine klare Scope-Regel: **keine beiläufigen Refactorings, keine hypothetischen Abstraktionen, keine „Verbesserungen“, die nicht für das Ziel nötig sind.** Anthropic warnt bei aktuellen Claude-Modellen explizit vor Overengineering und empfiehlt, Scope und Minimalität zu nennen.¹⁰

Bei Bugs ist ein besonders starkes Muster:

1. Bug reproduzieren.

⁹ Anthropic, “The AI-Native SDLC playbook,” Aug. 2026. <https://claude.com/blog/the-ai-native-sdlc-playbook>

¹⁰ Anthropic Docs, “Prompting best practices,” accessed Sep. 2026. <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/prompt-templates-and-variables>

2. Failing Test schreiben.
3. Test laufen lassen und erwarteten Fehler bestätigen.
4. Reproduktions-Test „einfrieren“.
5. Fix implementieren, ohne den Test anzupassen.
6. Relevante Suite ausführen.

Anthropic empfiehlt genau dieses Muster im AI-native SDLC und schlägt Hooks vor, um nach dem Commit eine Veränderung des Repro-Tests zu verhindern.¹¹

Phase D — Verification

Agentic Engineering funktioniert nur, wenn der Agent sein Ergebnis **selbst beobachten** kann. Gute Agenten brauchen ein geschlossenes Feedback-System:

- Unit-/Integration-/Contract-Tests.
- Typecheck und Lint.
- Build/Packaging.
- API-E2E-Tests.
- Browser-/Screenshot-Checks für UI.
- Logs, Traces, Metriken für Laufzeit-/Performance-Anforderungen.
- Datenbank-/Schema-Checks bei Migrationen.

OpenAIs Langzeit-Codex-Experiment nennt kontinuierliche Verifikation nach jedem Milestone als einen der entscheidenden Faktoren; die verwendeten Artefakte trennten Spec, milestone-basierten Plan, Runbook und Status/Audit-Log.¹² Das Harness-Engineering-Team ging noch weiter und machte UI, Logs, Metriken und Traces direkt agentenlesbar, sodass der Agent Bugs reproduzieren und Leistungsziele selbst prüfen konnte.¹³

Phase E — Unabhängiges Review

„Tests grün“ ist notwendig, aber nicht hinreichend. SWE-Gate zeigt für 2026 eine deutliche Lücke zwischen funktionalem Erfolg und Einhaltung von Review-Constraints: Von 644 Reparaturen, die funktionale Tests bestanden, verletzten 221 zusätzliche Review-Anforderungen.¹⁴ ABC-Bench und OmniCode zeigen ebenfalls, dass reale Backend-Lebenszyklen, Testgenerierung und andere Engineering-Aufgaben deutlich schwieriger bleiben als eng umrissene Patch-Benchmarks.¹⁵¹⁶

Daher sollte Review explizit gegen folgende Achsen laufen:

1. **Spec-Compliance:** Löst der Diff wirklich das definierte Problem?

¹¹ Anthropic, “The AI-Native SDLC playbook,” sections on feedback loops and continuous evals, Aug. 2026. <https://claude.com/blog/the-ai-native-sdlc-playbook>

¹² Derrick Choi, OpenAI Developers, “Run long horizon tasks with Codex,” 23 Feb. 2026. <https://developers.openai.com/blog/run-long-horizon-tasks-with-codex>

¹³ Ryan Lopopolo, OpenAI, “Harness engineering: leveraging Codex in an agent-first world,” 11 Feb. 2026. <https://openai.com/index/harness-engineering/>

¹⁴ Xin He et al., “SWE-Gate: Passing Functional Tests Is Not Enough for Software Engineering Agents,” arXiv:2609.04167, 3 Sep. 2026. <https://arxiv.org/abs/2609.04167>

¹⁵ Jie Yang et al., “ABC-Bench: Benchmarking Agentic Backend Coding in Real-World Development,” Findings of ACL 2026. <https://aclanthology.org/2026.findings-acl.1142/>

¹⁶ Atharv Sonwane et al., “OmniCode: A Benchmark for Evaluating Software Development Agents,” Findings of ACL 2026. <https://aclanthology.org/2026.findings-acl.2020/>

2. **Plan-Compliance:** Weicht die Implementierung sinnvoll oder unbemerkt ab?
3. **Correctness:** Bugs, Edge Cases, Nebenwirkungen.
4. **Security/Privacy:** Auth, Injection, Secrets, PII, Rechte.
5. **Architecture:** Layering, Abhängigkeiten, öffentliche Verträge.
6. **Operability:** Logging, Failure Modes, Migration, Rollback.
7. **Simplicity:** Unnötige Abstraktion, Dead Code, Scope Creep.

Empfehlung für Codex + Claude: Implementierung und finalen Review möglichst nicht in derselben langen Session durchführen. Entweder frischen Kontext oder den anderen Modell-/Tool-Stack nutzen. Das ist keine Garantie gegen Fehler, reduziert aber korrelierte blinde Flecken und zwingt zu einer eigenständigen Rekonstruktion der Anforderungen.

4. Context Engineering: weniger laden, besser navigieren

Kontext ist eine knappe Ressource. Ein Agent soll nicht „alles wissen“, sondern **wissen, wo er nachsehen muss**.

Gute Muster

1. **Kleine Root-Datei, lokale Details.** Claude Code lädt Root- und passende Unterverzeichnis-`CLAUDE.md` additiv; Anthropic empfiehlt lokale Test-/Build-Kommandos und lokale Konventionen in den jeweiligen Subtrees.¹⁷
2. **Repo-Karten statt Volltext.** Ein `ARCHITECTURE.md` mit Top-Level-Domains, Layern und Einstiegspunkten ist oft wertvoller als hunderte Seiten prose.
3. **Code Intelligence vor Grep-Flut.** In großen Codebases empfiehlt Anthropic LSP/Symbolsuche, weil String-Grep zu viele irrelevante Treffer in den Kontext zieht.¹⁸
4. **Generierte/irrelevante Dateien ausblenden.** Build-Artefakte, vendored code und große generated files erhöhen Suchkosten und Rauschen.
5. Session-State ins Dateisystem verschieben. Bei längeren Aufgaben sollte der relevante Task-State in `docs/exec-plans/active/<task>.md` liegen, nicht nur im Chat-Kontext. Für einen sequenziellen Workflow reicht normalerweise eine Datei mit Goal, Plan, Trade-offs/Decisions, Progress, Failed approaches, Learnings und Next step. Eine neue Session kann genau diese Datei lesen und fortsetzen. Git-Commits bleiben die technische Historie; dauerhafte Architektur- oder Produktwahrheit wird nach Abschluss in die kanonischen Docs überführt.

Context Debt

Jede Information, die nur in Chat, Slack oder Köpfen existiert, ist für den nächsten Agenten praktisch verloren. OpenAI beschreibt genau das als „agent legibility“-Problem.¹⁹ Eine wiederkehrende Regel oder Entscheidung sollte deshalb möglichst in ein versioniertes, auffindbares Artefakt überführt werden.

¹⁷ Anthropic, “How Claude Code works in large codebases: Best practices and where to start,” 2026.
<https://claude.com/blog/how-claude-code-works-in-large-codebases-best-practices-and-where-to-start>

¹⁸ Anthropic, “How Claude Code works in large codebases: Best practices and where to start,” 2026.
<https://claude.com/blog/how-claude-code-works-in-large-codebases-best-practices-and-where-to-start>

¹⁹ Ryan Lopopolo, OpenAI, “Harness engineering: leveraging Codex in an agent-first world,” 11 Feb. 2026.
<https://openai.com/index/harness-engineering/>

5. Skills, Rules, Hooks, MCP und Scripts richtig trennen

Skills: wenige, hochwertige Workflows

Skills sind nützlich, aber nicht der Kern des Harness. Sie lohnen sich vor allem für wiederkehrende, mehrstufige Abläufe, bei denen jedes Mal dieselbe Reihenfolge, dieselben Quellen und dieselben Checks gebraucht werden. Für einen einzelnen Engineer würde ich bewusst nur mit drei Skills starten und einen vierten erst bei Bedarf ergänzen:

- `/plan-from-spec` — Spec und Repo lesen, bei externen Abhängigkeiten das Freshness Gate ausführen und einen konkreten Plan erstellen; noch keinen Code ändern.
- `/debug-fix` — Bug reproduzieren, failing Regression-Test erzeugen, minimal beheben und die relevante Verification ausführen.
- `/independent-review` — in frischem Kontext Spec + Diff prüfen, Findings belegen und erst danach den ursprünglichen Plan als Deviation-Audit lesen.
- `/release-check (optional)` — Release-/Migrationscheckliste ausführen, wenn Deployments, Changelogs oder Compatibility-Schritte im Repo tatsächlich komplex und wiederkehrend sind.

Wichtig: Nicht präventiv zehn weitere Skills bauen. Wenn ein Ablauf erst einmal vorkommt, reicht ein normaler Prompt. Erst wenn derselbe mehrstufige Prozess wiederkehrt und dabei tatsächlich Konsistenz verloren geht, wird er zum Skill.

OpenAI nutzt Repo-Skills in den Agents-SDK-Repositories und berichtet, dass der Workflow aus `AGENTS.md`, `.agents/skills/`, optionalen Scripts und GitHub Actions wiederkehrende Wartungsarbeit standardisiert.²⁰ Der entscheidende Mechanismus ist progressive disclosure: Metadaten zuerst, vollständige Skill-Anweisung nur bei Auswahl, zusätzliche Referenzen/Scripts erst bei Bedarf.²¹

Rules: dauerhafte Constraints

Rules sind gut für Invarianten, die nicht rein mechanisch formulierbar sind, z. B.:

- API-Vertrag muss rückwärtskompatibel bleiben.
- Bestimmte Daten dürfen nie geloggt werden.
- Ein spezieller Service darf nur über Provider X auf Ressource Y zugreifen.

OpenAI empfiehlt für Codex Code Review kleine, eng gescopete Regelsets mit explizitem „safe path“ und lässt mechanische Style-Checks weiterhin in CI.²²

Hooks / Scripts: deterministische Automatisierung

Anthropic trennt Hooks klar von Prompt-Instruktionen: Sie feuern deterministisch bei Events wie Tool Calls, File Edits oder Session Start und eignen sich für Linting, blockierte Kommandos, Kontext-Injektion oder Stop-Gates.²³

Ein gutes Heuristik-Schema:

²⁰ Kazuhiro Sera, OpenAI Developers, “Using skills to accelerate OSS maintenance,” 9 Mar. 2026. <https://developers.openai.com/blog/skills-agents-sdk>

²¹ Kazuhiro Sera, OpenAI Developers, “Using skills to accelerate OSS maintenance,” 9 Mar. 2026. <https://developers.openai.com/blog/skills-agents-sdk>

²² Hari Srikanth, OpenAI Developers, “Custom Code Review rules for Codex,” 20 Jul. 2026. <https://developers.openai.com/blog/custom-code-review-rules-for-codex>

²³ Michael Segner, Anthropic, “Steering Claude Code: when to use CLAUDE.md, skills, hooks, and subagents,” 18 Jun. 2026. <https://claude.com/blog/steering-claude-code-skills-hooks-rules-subagents-and-more>

Wenn du sagen willst...	Dann nutze...
„Claude/Codex soll immer wissen, dass...“	CLAUDE/AGENTS
„Wenn Dateien unter X betroffen sind, gilt...“	path rule / nested instructions
„Bei Task-Typ Y führe Prozess Z aus...“	Skill
„Nach jedem Edit muss sicher A passieren...“	Hook/Script
„Der Agent braucht Zugriff auf System B...“	MCP/native integration
„Die Änderung darf nicht gemerged werden, wenn...“	CI/branch protection

MCP: Kontext und Aktionen, nicht einfach „mehr Tools“

MCP lohnt sich, wenn der Agent sonst blind gegenüber wichtigen Systemen wäre: Ticketing, Doku, Datenbanken, Observability, interne APIs. Die Tooloberfläche sollte klein, semantisch klar und möglichst eng berechtigt sein. OpenAIs Codex-Harness trennt bewusst Host-Anwendung, Business Context/Tools und Agent-Loop; consequential writes können approvalpflichtig bleiben.²⁴

6. Codex vs. Claude Code: sinnvolle Rollenverteilung 2026

Die Feature-Landschaften überlappen inzwischen stark; eine harte „Tool A kann X, Tool B nicht“-Trennung altert schnell. Sinnvoller ist eine Einteilung nach dem aktuellen Produkt-Schwerpunkt.

Bereich	Codex	Claude Code
Persistente Repo-Instruktionen	AGENTS.md, nested scope	CLAUDE.md, path-scoped rules
Wiederverwendbare Workflows	Skills, Scripts, GitHub Actions	Skills, Commands, Hooks
Long-horizon	explizite Langzeit-Runs, durable project memory, Cloud/remote flows	lange Sessions, compaction/state files, dynamic workflows
Review	lokales/Cloud Code Review + custom repo rules	Review-Agents/CI + REVIEW.md-artige Policies
Eigene Integration	offener CLI/SDK/App-Server-Harness	Claude Agent/Code-Ökosystem, MCP/Plugins
Stärkster strategischer Fit	Agent-Loop in Produkt/CI einbetten	lokale interaktive Engineering-Workflows und umfangreiche Agent-Customization

Empfohlenes Dual-Agent-Muster

Nicht: Jede Aufgabe gleichzeitig an beide schicken.

Sondern:

- Primär-Agent implementiert.
- Zweit-Agent übernimmt frischen Review bei riskanten/komplexen Änderungen.
- Bei Architekturfragen: beide unabhängig planen lassen; Mensch wählt die bessere Richtung, danach nur ein Implementer.
- Für repetitive Task-Klassen: über Evals messen, welches Setup auf dem eigenen Repo zuverlässiger ist.

Damit wird das Modell austauschbarer. Der eigentliche Wettbewerbsvorteil liegt in Repo-Struktur, Feedback-Loops und Evals.

²⁴ Nicolas Bonamy & Derrick Choi, OpenAI Developers, “Codex as a platform: build on the open agent harness,” 19 Aug. 2026. <https://developers.openai.com/blog/codex-as-a-platform>

7. Spätere Reifestufe: Evals erst bei echtem Bedarf

Wenn AGENTS.md, CLAUDE.md, Skills, Hooks oder Agent-Routing geändert werden, ändert man faktisch die Software, die den Agenten steuert. Diese Änderungen sollten deshalb Regressionstests bekommen.

OpenAI empfiehlt für Skills explizite Evals mit Fragen wie: Wurde der Skill ausgelöst? Wurden erwartete Commands ausgeführt? Entspricht der Output den Konventionen?²⁵ Anthropic empfiehlt im SDLC-Playbook, 20–50 reale Aufgaben aus jüngster Arbeit als Eval-Set zu verwenden und das Set bei Änderungen an CLAUDE.md, Skills oder Hooks automatisch auszuführen.²⁶

Minimaler Eval-Datensatz

Pro Task speichern:

```
id: bug-api-017
prompt: "...
fixture: "repo snapshot / commit"
expected:
  tests: pass
  must_touch:
    - src/api/...
  must_not_touch:
    - migrations/...
  review_constraints:
    - "public field remains backward compatible"
metrics:
  human_interventions: 0
  max_runtime_minutes: 30
```

Sinnvolle Team-Metriken

- First-pass CI success rate.
- Rework-Loops bis Merge.
- Anzahl menschlicher Interventionen pro Task.
- Review-Findings nach Agent-Abschluss.
- Revert-/Change-failure-rate.
- Zeit von akzeptiertem Plan bis merge-ready.
- Kosten/Tokens pro erfolgreich abgeschlossenem Task.
- Anteil von Tasks, die ohne manuellen Code-Edit landen.

Nicht „Tokenverbrauch“ oder „Anzahl Agenten-Sessions“ optimieren, sondern **erfolgreiche, reviewbare Outcomes pro menschlicher Aufmerksamkeit**.

Die aktuelle Produktivitätsforschung mahnt zur Vorsicht bei pauschalen Speedup-Zahlen. METR hält seine neueren frühen-2026-Daten wegen Selektionseffekten für nur schwach aussagekräftig, sieht aber Hinweise darauf, dass aktuelle Tools gegenüber 2025 nützlicher geworden sind.²⁷ Anthropic findet in einer Analyse von rund 400.000

²⁵ Dominik Kundel & Gabriel Chua, OpenAI Developers, “Testing Agent Skills Systematically with Evals,” 22 Jan. 2026. <https://developers.openai.com/blog/eval-skills>

²⁶ Anthropic, “The AI-Native SDLC playbook,” sections on feedback loops and continuous evals, Aug. 2026. <https://claude.com/blog/the-ai-native-sdlc-playbook>

²⁷ METR, “We are Changing our Developer Productivity Experiment Design,” 24 Feb. 2026. <https://metr.org/blog/2026-02-24-uplift-update/>

Claude-Code-Sessions gleichzeitig, dass menschliche Expertise weiterhin positiv mit Erfolg korreliert und Menschen typischerweise mehr Planungsentscheidungen, Claude mehr Ausführungsentscheidungen treffen.²⁸ Agentic Engineering verschiebt Expertise also eher auf **Spezifikation, Systemdesign und Review** als dass sie verschwindet.

8. Die häufigsten Anti-Patterns

1. Eine riesige AGENTS.md / CLAUDE.md

Kontext wird dauerhaft verbrannt, Regeln konkurrieren, Doku veraltet. Besser: Root als Karte, Details in Docs/Skills/Rules.

2. Agent ohne ausführbare Definition von „done“

„Mach Feature X“ ohne Tests, E2E oder visuelle Prüfung führt zu plausibel aussehendem, aber unbewiesenem Output.

3. Tests als einzige Spezifikation

Agenten können Tests überfitten oder Review-/Architekturconstraints verletzen. Spec + Review Policy müssen separat existieren.²⁹

4. „Try harder“-Prompts bei wiederholtem Fehler

Wenn ein Agent denselben Fehler mehrfach macht, fehlt typischerweise ein Harness-Asset: Doku, Rule, Tool, Test, Linter oder Skill.³⁰

5. Mensch kopiert ständig Kontext in den Chat

Der Agent sollte Repo, Issue, Logs, CI und relevante Tools direkt lesen können. Copy-paste ist ein Signal für fehlende Integration.

6. Autonomie ohne Reversibilität

Lokale, reversible Aktionen können weitgehend autonom laufen. Shared Infrastructure, Push/Force, Datenlöschung, externe Kommunikation oder produktive Writes brauchen klar definierte Approval-Grenzen.^{31,32}

²⁸ Anthropic Research, “Agentic coding and persistent returns to expertise,” 16 Jun. 2026. <https://www.anthropic.com/research/claude-code-expertise>

²⁹ Xin He et al., “SWE-Gate: Passing Functional Tests Is Not Enough for Software Engineering Agents,” arXiv:2609.04167, 3 Sep. 2026. <https://arxiv.org/abs/2609.04167>

³⁰ Ryan Lopopolo, OpenAI, “Harness engineering: leveraging Codex in an agent-first world,” 11 Feb. 2026. <https://openai.com/index/harness-engineering/>

³¹ Nicolas Bonamy & Derrick Choi, OpenAI Developers, “Codex as a platform: build on the open agent harness,” 19 Aug. 2026. <https://developers.openai.com/blog/codex-as-a-platform>

³² Anthropic Docs, “Prompting best practices,” accessed Sep. 2026. <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/prompt-templates-and-variables>

9. Ein pragmatisches Setup für einen einzelnen Engineer

Für einen typischen professionellen Python/TypeScript/Data/LLM-Workflow würde ich mit einem einfachen, sequenziellen Agent-Loop starten. Das folgende Setup liefert wahrscheinlich den größten Nutzen pro Aufwand.

Stufe 1 — 1 Nachmittag

- Root AGENTS.md und CLAUDE.md auf ca. 80–150 Zeilen bringen.
- ARCHITECTURE.md mit 1–2 Seiten Systemkarte anlegen.
- Standardbefehle setup, test-fast, test, lint, typecheck vereinheitlichen.
- Definition von „done“ explizit machen.

Stufe 2 — nächste 1–2 Wochen

- Für komplexe Tasks: Spec + eine task-lokale Exec-Plan-Datei standardisieren (Plan, Trade-offs/Decisions, Progress, Failed approaches, Learnings, Next step).
- Bugfix: failing test first.
- Drei kleine Skills anlegen: /plan-from-spec, /debug-fix und /independent-review.
- Einen unabhängigen Review-Workflow definieren.
- UI/API/ETL-spezifische E2E-Checks ergänzen.

Stufe 3 — wenn Wiederholungen sichtbar werden

- Weitere Skills nur ergänzen, wenn derselbe mehrstufige Ablauf mehrfach wiederkehrt und Standardisierung wirklich Zeit oder Fehler spart.
- Deterministische Regeln in Hooks/Linter/CI verschieben.
- MCP nur für Systeme hinzufügen, die der Agent wirklich braucht.
- Evals erst ergänzen, wenn wiederkehrende Task-Klassen oder Harness-Änderungen systematisch verglichen werden sollen; dann mit wenigen realen Aufgaben starten und schrittweise erweitern.

10. Copy-ready Prompt für komplexe Engineering-Tasks

Goal

- <konkretes beobachtbares Ziel>

Context

- Read AGENTS.md / CLAUDE.md and the referenced architecture/docs first.
- Relevant issue/spec: <path/link>
- Existing pattern to follow: <file/module>

Scope

- In scope: ...
- Out of scope: ...

Constraints

- Preserve ...
- Do not change ...
- Prefer the smallest implementation that satisfies the requirements.

Operating mode

1. Investigate the relevant code before making claims or edits.
2. Freshness gate: if the task depends on an external library, framework, API, SDK or cloud-service behavior, identify the exact version used by the repo and consult official version-specific documentation before finalizing the plan. Record the relevant source/version; do not upgrade dependencies unless required by the spec.
3. Produce an implementation plan naming files, order, risks, and verification. Surface every material trade-off explicitly: realistic options, key pros/cons, and a recommendation with rationale. Do not invent artificial alternatives.
4. For non-trivial changes, save the accepted plan to docs/exec-plans/active/<task>.md and keep the same file current with sections for Trade-offs/Decisions, Progress, Failed approaches, Learnings, and Next

```

step.
5. Implement the plan with minimal unrelated changes.
6. Verify using the repo's required checks. For bugs, reproduce with a failing test first.
7. Review your final diff against the spec, plan, architecture rules, security, and backward compatibility.
8. Before closing the task, classify the Learnings: keep task-specific history in the exec plan; promote only confirmed, generalizable learnings into the appropriate permanent repo artifact or executable guardrail.
9. Report evidence: tests/checks run, relevant outputs, and any unresolved risks.

Acceptance criteria
- ...
- ...

```

Der Prompt ist absichtlich nicht extrem lang. Die eigentliche Detailtiefe soll aus dem Repo, dem Spec und den Skills kommen. Das ist nachhaltiger als jedes Mal einen riesigen „Master Prompt“ zu bauen.

11. Schlussfolgerung

Der produktivste Umgang mit Codex und Claude Code sieht zunehmend wie klassisches gutes Software Engineering aus — nur eine Ebene höher. Gute Architektur, klare Verträge, kleine Interfaces, reproduzierbare Umgebungen, Tests, Observability und Versionierung werden wichtiger, nicht weniger wichtig.

Die zentrale Einheit ist nicht mehr der Prompt. Es ist der **Agentic Engineering Loop**:

```

Intent → Spec → Explore + Plan → Implement in isolation → Verify → Independent review
      ↑                                     ↓
      |                                     |
Harness aus Fehlern verbessern ← Observe / Merge ←—————|

```

Wer diesen Loop baut, kann Modelle austauschen und zunehmend autonom arbeiten lassen. Wer nur Prompts optimiert, bleibt von einzelnen guten Sessions abhängig.